Robotics Agent Project

Seokmin Lee

2026.08.04

contents

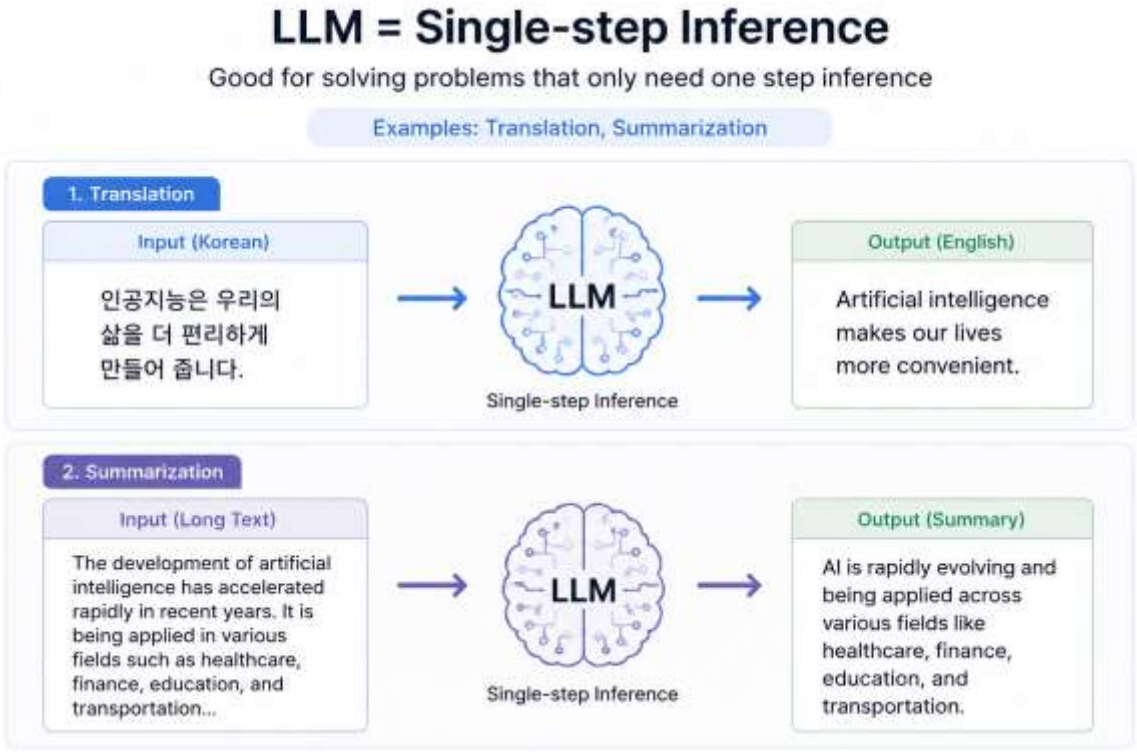
- Introducing Concept of Agentic AI (LLM)
  - Why Agentic AI?
  - Design Agentic AI
  - Fun Project : Finding Dating Places
- Possible Design Choices for Robotic Agents
- Research Direction

Introducing Concept of Agentic AI

Introducing Concept of Agentic AI
- Why Agentic AI? -

Why Agentic AI? LLM is not enough?

- LLM is basically Single-step Inference
  - Good for solving problems that only need one step inference
    - translation, summarization



Why Agentic AI? LLM is not enough?

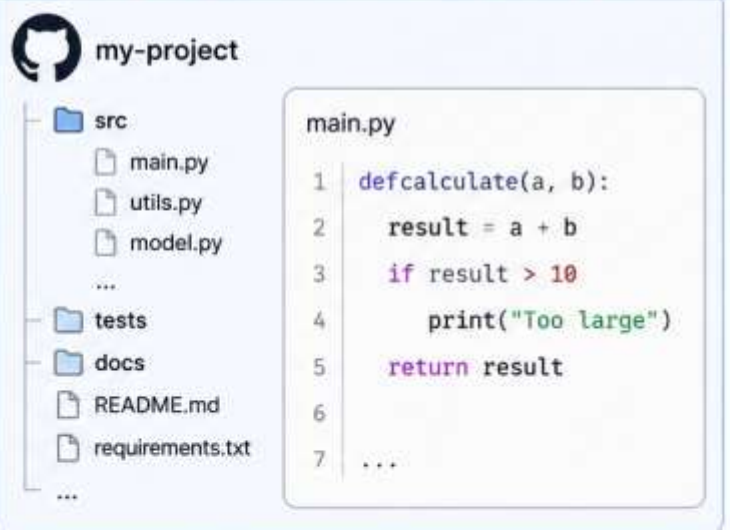
- But, What About Real-World Problems?
  - **Example : “Fix the bug in this GitHub project”**
  - Naïve approach
    - Input : Entire GitHub repository
    - Output : The bug fix
  - Is this really the best approach?

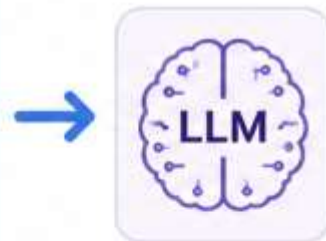
But, What About Real-World Problems?

⚠ Example: "Fix the bug in this GitHub project."

Naive approach

Input: Entire GitHub repository





Output: The bug fix

main.py (fixed)

```
1 defcalculate(a, b):
2   result = a + b
3   if result > 10:
4     print("Too large")
5   return result
6
7 ...
```

? Is this really the best approach?

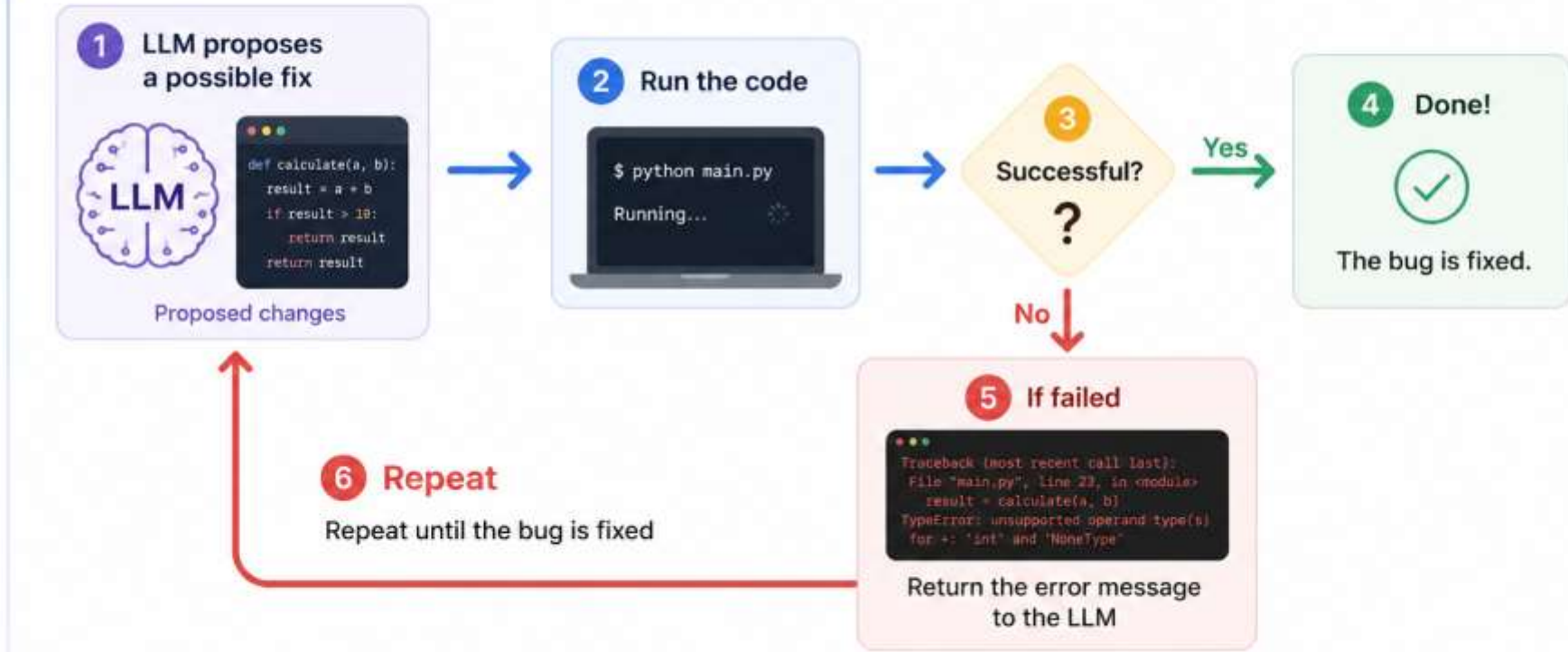
Why Agentic AI? LLM is not enough?

- But, What About Real-World Problems?
  - **Example : “Fix the bug in this GitHub project”**
- A better Approach
  1. LLM proposes a possible fix
  2. Run the code
  3. If successful -> Done
  4. If failed -> Return the error message to the LLM
  5. Repeat until the bug is fixed

But, What About Real-World Problems?

Example: "Fix the bug in this GitHub project"

A Better Approach

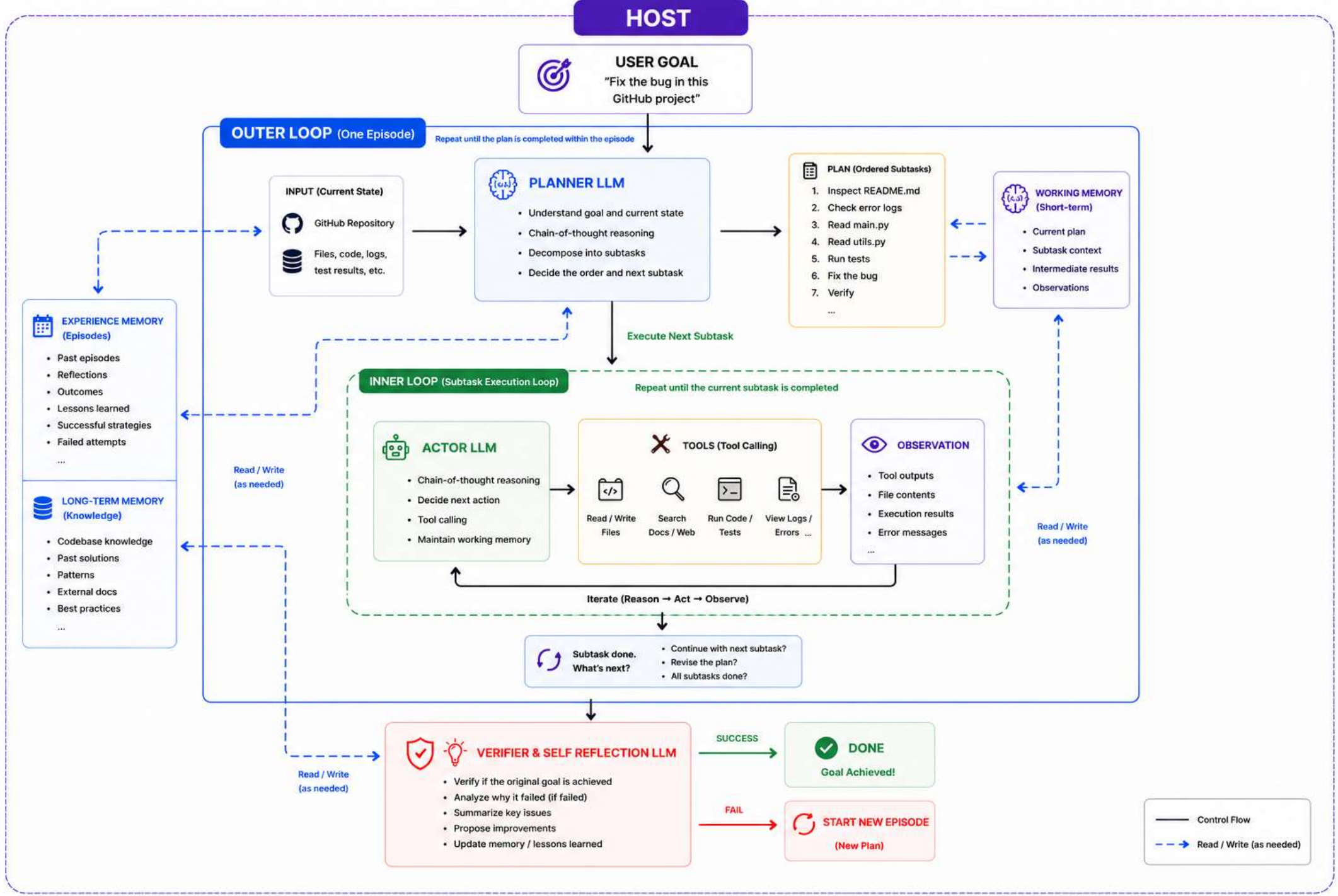


Real-world problem solving requires iterative reasoning and feedback.

“LLMs generate responses,
whereas agents complete tasks”

“Building an AI agent means creating a system that continuously tries, learns, and retries until the task is successfully completed”

Introducing Concept of Agentic AI
- Design Agentic AI -


HOST
USER GOAL
"Fix the bug in this GitHub project"
OUTER LOOP (One Episode)
Repeat until the plan is completed within the episode
INPUT (Current State)
GitHub Repository
Files, code, logs, test results, etc.
PLANNER LLM
Understand goal and current state
Chain-of-thought reasoning
Decompose into subtasks
Decide the order and next subtask
PLAN (Ordered Subtasks)
1. Inspect README.md
2. Check error logs
3. Read main.py
4. Read utils.py
5. Run tests
6. Fix the bug
7. Verify
...
WORKING MEMORY (Short-term)
Current plan
Subtask context
Intermediate results
Observations
EXPERIENCE MEMORY (Episodes)
Past episodes
Reflections
Outcomes
Lessons learned
Successful strategies
Failed attempts
...
LONG-TERM MEMORY (Knowledge)
Codebase knowledge
Past solutions
Patterns
External docs
Best practices
...
Read / Write (as needed)
Execute Next Subtask
INNER LOOP (Subtask Execution Loop)
Repeat until the current subtask is completed
ACTOR LLM
Chain-of-thought reasoning
Decide next action
Tool calling
Maintain working memory
TOOLS (Tool Calling)
Read / Write Files
Search Docs / Web
Run Code / Tests
View Logs / Errors ...
OBSERVATION
Tool outputs
File contents
Execution results
Error messages
...
Read / Write (as needed)
Iterate (Reason → Act → Observe)
Subtask done. What's next?
Continue with next subtask?
Revise the plan?
All subtasks done?
Read / Write (as needed)
VERIFIER & SELF REFLECTION LLM
Verify if the original goal is achieved
Analyze why it failed (if failed)
Summarize key issues
Propose improvements
Update memory / lessons learned
SUCCESS
DONE
Goal Achieved!
FAIL
START NEW EPISODE
(New Plan)
Control Flow
Read / Write (as needed)

Agentic AI – Core Elements (1/2)

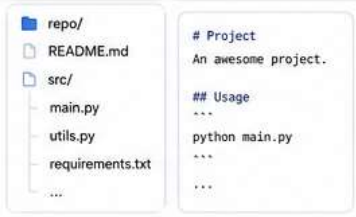
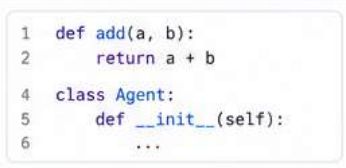
Element	Description
 **Goal**	Defines the objective to be achieved. It serves as the starting point for all decision-making.
 **Planning**	Decomposes the goal into multiple executable sub-tasks and determines the order in which they should be performed.
 **Perception**	Observes and interprets the current state of the external environment through inputs such as cameras, text, code, or logs.
 **Memory**	Stores previous observations, execution results, and important information, and retrieves them when needed.
 **World Model** (Internal State)	An internal representation of the current environment, constructed by integrating current information from perception with past information in memory. Subsequent reasoning is based on this world model.

Agentic AI – Core Elements (2/2)

Element	Description
 **Reasoning**	Interprets the current situation based on the world model and determines the next action. If necessary, it revises the existing plan.
 **Tool Use**	Interacts with external tools such as search engines, Python, Git, web browsers, or robot controllers.
 **Action**	Executes actions in the environment, such as modifying code, calling APIs, or controlling a robot.
 **Observation**	Monitors how the environment changes after an action is executed.
 **Reflection**	Evaluates the outcome, analyzes the causes of failures, and updates future strategies accordingly.

Perception: From Raw Inputs to Representations

Perception converts diverse external inputs into structured representations that the agent can understand.

External Input (Raw Data)	What It Means	Encoder / Processor	Resulting Representation	Feeds into World Model (Internal State)
GitHub Repository 	The entire repository (code, README, config, docs, etc.) is read and parsed.	–	Text / Code (as is)	All representations are integrated to form the agent's internal view of the environment.
Image (e.g., RGB) 	The image is encoded into a compact feature representation capturing its visual semantics.	**Image Encoder** (e.g., DINOv2, MAE, ViT)	**Image Representation** (Patch / Global Embeddings)	
Video (e.g., MP4) 	The video is encoded into spatiotemporal features capturing motion and context.	**Video Encoder** (e.g., VideoMAE, InternVideo, ViT)	**Video Representation** (Spatiotemporal Embeddings) time →	
Natural Language (e.g., instruction) `Find the bugs in this code and explain how to fix them.`	Natural language is provided as text input.	–	Text (as is)	
Code (e.g., Python) 	Code is provided as text input.	–	Code (as is)	

Key Idea: Perception turns raw, diverse inputs into a unified, structured form.
Images and videos are encoded into embeddings; natural language and code are used as text.

Planning: From Goal to Actionable Sub-tasks

Planning decomposes the user's goal into executable sub-tasks and determines the order of execution.
The level of decomposition depends on task complexity.

User Goal (Example)	What It Means	Sub-task Planning (Output)	Explanation
1 방을 치워라 	The room is messy. Break the goal into multiple steps that lead to a clean and organized room.	1 Trash collection — Pick up trash and throw it away. → 2 Collect items — Gather items that are scattered around. → 3 Sort items — Sort items into categories (clothes, books, etc.). → 4 Organize & put away — Put items in their proper places. → 5 Arrange surfaces — Arrange things on desk, bed, and other surfaces. → 6 Final check — Check the room and make final adjustments.	Complex task. Decomposed into ordered, executable sub-tasks.
2 건물에서 석민이를 찾아라 	Find Seokmin in a building. Requires systematic search through areas and floors.	1 Analyze the building — Check the building layout, floors, and access points. → 2 Go to target floor — Move to the floor where Seokmin is likely to be. → 3 Search each area — Search rooms and areas on the floor one by one. → 4 Check people — Identify and check people in the area. → 5 Verify identity — Confirm if the person is Seokmin. → 6 Report result — Report the location or that he was not found.	Complex task. Decomposed into ordered, executable sub-tasks to ensure complete search.
3 컵을 잡아라 	A simple task. It can be executed directly without decomposition.	컵을 잡아라 (Grab the cup) → Execute directly	Simple task. No decomposition needed. Execute directly.

Key Idea: Planning bridges the gap between high-level goals and low-level actions by breaking goals into the right level of detail.

Memory: Structured Storage for Past and Present

Memory enables the agent to store, organize, and retrieve useful information across different time scales and purposes.

	Experience Memory (Episodic Memory)	Long-term Memory (Knowledge, Semantic) (Semantic Memory)	Working Memory (Short-term / Active Memory)
Purpose	과거의 경험(에피소드)을 저장하고, 나중에 회상하여 유사한 상황에서 활용한다.	일반적이고 지속적인 지식(사실, 개념, 규칙 등)을 체계적으로 저장하고 활용한다.	현재 수행 중인 한 task의 한 episode(또는 trial)를 해결하는 동안 필요한 장보를 임시로 유지한다.
What It Stores	**중요한 에피소드(경험)** • 완료한 작업의 경과와 결과 • 성공/실패 경험과 그 원인 • 상호작용 기록 (관찰, 행동, 보상 등) • 중요한 사건이나 결정의 맥락	**의미(semantic) 있는 지식** • 개념, 사실, 정의 • 규칙, 절차, 일반 원리 • 도메인 지식, 상식 • 정리된 요약 정보, 임베딩 형태의 지식	**현재 task에 필요한 정보** • 현재 목표와 서브태스크 • 지금까지의 관찰과 추론 결과 • 계획 및 중간 결과 (변수, 계산 동) • 최근 대화/도구 사용 내역 등
Time Scale	과거 → 현재 장기간 (여러 에피소드에 걸쳐)	오래 지속 (지속적이며 일반화된 지식)	episode / trial 시작 → episode / trial 종료 단기 (현재 episode/trial 동안)
Example	"지난주에 사용자에게 코드 버그를 수정해줬고 성공했어. 비슷한 상황에서 같은 방법이 유용했지." (에피소드 단위로 저장)	"Python에서 리스트 컴프리헨션 문법은 `[expr for item in list if cond]` 형식이야." (지식 단위로 저장)	"지금 사용자가 요청한 파일을 열었고, 오류 로그를 확인 중이야. 다음으로는 원인 분석을 해야 해." (현재 task 해결을 위해 유지)
Key Point	보통 중요한 에피소드 등을 담음 (경험의 기록과 맥락 보존)	Semantic한 지식 등을 정리해서 담음 (일반화 가능한 지식 저장)	한 task에 대한 한 episode(또는 trial)를 풀고 있을 때 유지하는 메모리 (임시적이고 작업 지향적)

How They Work Together

Experience Memory (과거의 경험 제공) → Long-term Memory (Knowledge, Semantic) (지식과 배경 제공) → Working Memory (현재 문제 해결에 활용) → Reasoning & Action

핵심 요약

경험은 사례를, 지식은 원리를,
작업 기억은 현재의 맥락을 담아
에이전트의 지능적인 행동을 지원한다.

World Model (Internal State): An Agent's Internal View of the World

World Model은 Perception을 통해 얻은 현재 정보와 Memory에 저장된 과거 정보를 결합하여 환경을 내부적으로 표현한 상태(State)입니다.
이 모델을 기반으로 Reasoning, Planning, Prediction 등이 이루어집니다.

1. Inputs

Perception (현재 정보)

외부 환경으로부터 관찰한 현재 정보

이미지 · 비디오 · 자연어 · 코드 / 로그 · ...

+

Memory (과거 정보)

과거의 경험과 지식

- **Experience Memory**: 중요한 에피소드, 경험 기록
- **Long-term Memory (Semantic)**: 지식, 규칙, 개념 등 의미적 정보
- **Working Memory**: 현재 수행 중인 작업의 정보

2. World Model (Internal State)

다양한 정보를 통합하여 내부적으로 표현한 환경의 상태

- **객체 (Objects)**: - 소파 - 테이블 - 컵 - 책 ...
- **속성 (Attributes)**: - 위치, 크기 - 색상, 상태 - 온/오프 ...
- **관계 (Relations)**: - 위에 있음 - 옆에 있음 - 들고 있음 ...

- **공간 정보** (위치, 지도)
- **시간 정보** (변화, 순서)
- **상태 정보** (상황, 맥락)
- **목표/의도** (현재 목표, 제약 조건)

Unified Internal Representation
구조화된 표현 (그래프, 백터, 토큰 등) ...

3. Used For

World Model을 기반으로 다양한 고차원적 인지 및 의사결정을 수행

- **Reasoning**: 현재 상황을 이해하고 다음 행동을 결정
- **Planning**: 계획 수립 및 수정
- **Prediction**: 미래 상태를 예측하고 결과를 시뮬레이션
- **Action Selection**: 최적의 행동을 선택하고 실행

...

주요 특징 (Characteristics)

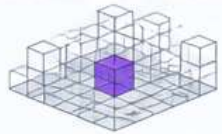
- **통합 (Integration)**: 다양한 모달리티와 시간 스케일의 정보를 하나로 통합
- **압축 (Compression)**: 복잡한 원시 정보를 핵심적인 구조로 압축하여 표현
- **동적 (Dynamic)**: 새로운 관찰과 경험으로 지속적으로 업데이트 (Online Learning)
- **추론 친화적 (Reasoning-friendly)**: 고차원 추론과 의사결정에 효과적으로 활용될 수 있는 구조로 구성

예시 (Example)

관찰 (Perception)
로봇이 테이블 위의 컵을 카메라로 관찰

→ **기억 (Memory)**
이전 경험: 컵은 보통 잡을 수 있고, 손잡이가 오른쪽에 있음

→ **World Model (Internal State)**
컵의 위치, 크기, 손잡이 방향, 테이블의 상태 등을 내부적으로 표현



→ **활용 (Reasoning/Action)**
컵을 잡기 위한 최적의 움직임을 계획하고 실행

핵심 요약 World Model은 에이전트가 환경을 '이해'하고 '예측'하며 '계획'할 수 있게 해주는 내부적인 정신 모델입니다.

Reasoning: Thinking to Decide What to Do Next

Reasoning is the core cognitive process that interprets the world model, evaluates options, and decides the next step. It happens in the planning stage and the action selection stage.

1. Where Reasoning Happens

Reasoning은 주로 Planner와 Action 단계에서 수행된다.

Planner에서의 Reasoning

목표를 달성하기 위한 최적의 하위 과제와 순서를 결정하기 위해 Reasoning을 수행한다.

Goal (방을 치워라) → Reasoning (계획 수립) → Sub-task Plan
1. 쓰레기 수거
2. 물건 분류
3. 정리 및 배치
4. 최종 점검

고려하는 정보: 현재 상태 (World Model) · 과거 경험 (Experience) · 지식/규칙 (Knowledge) · 제약 조건 (Constraints)

Action에서의 Reasoning

현재 상태에서 어떤 행동이 가장 적절한지 판단하고 구체적인 행동을 선택한다.

Current State → Reasoning (행동 선택) → Action (컵을 잡아라)

고려하는 정보: 현재 관찰 (Observation) · 목표/계획 (Goal/Plan) · 환경 변화 예측 (Prediction) · 안전/계약 (Safety/Constraints)

2. How Reasoning Works: Chain of Thought (CoT)

Reasoning은 Chain of Thought와 같이 단계적으로 사고를 전개하여 결론(다음 행동 또는 계획)을 도출한다.

예시 Task: 건물에서 석민이를 찾아라

Step 1	**이해 (Understand)** 목표는 건을 내에서 석민이를 찾는 것이다.	문제 이해
Step 2	**분석 (Analyze)** 건물 구조, 층별 정보, 출입구, 사람 분포 등을 확인해야 한다.	정보 분석
Step 3	**추론 (Infer)** 석민이가 있을 법한 위치(사무실, 회의실 등)를 우선 순위로 추론한다.	가설 수립
Step 4	**계획 (Plan)** 우선 순위가 높은 층/구역부터 탐색 순서를 계획한다.	계획 수립
...		...
Conclusion	**결론 (Decide)** 3층 → 회의실 → 좌측 사무실 순으로 탐색하고, 발견 시 신원 확인 후 보고한다.	다음 행동 결정

이러한 단계적 사고(Chain of Thought)를 통해 더 정확하고 신뢰할 수 있는 결정을 내릴 수 있다.

Key Takeaways

1. 언제 수행하나?

Reasoning은 주로 Planner(계획 수립)와 Action(행동 선택)에서 수행된다.

2. 어떻게 수행하나?

Chain of Thought와 같은 단계적 사고를 통해 정보를 해석하고 결론을 도출한다.

3. 왜 중요한가?

복잡한 문제를 체계적으로 해결하고, 더 나은 계획과 행동을 선택할 수 있게 한다.

핵심 요약

Reasoning은 "생각"을 통해 현재 상황을 이해하고, 다음 계획과 행동을 결정하는 에이전트의 지능적 핵심 과정이다.

Tool Use: Extending LLMs Beyond Their Native Abilities

Tool Use는 LLM이 직접 수행할 수 없는 작업을 외부 도구(Tool)를 활용하여 해결할 수 있게 해준다.

1. Why Use Tools?

LLM은 텍스트 생성에 강하지만,
실시간 정보 조회, 계산, 파일 접근, 외부 시스템 제어 등은
직접 수행할 수 없다.

→ 이러한 한계를 보완하기 위해 Tool을 이용한다.

LLM이 잘하는 것	LLM만으로는 어려운 것 (Tool로 해결)
자연어 이해 및 생성	실시간 웹 검색
요약, 번역, 분석, 추론	정확한 수치 계산
계획 수립, 의사결정 지원	파일 읽기/쓰기, 데이터 접근
	외부 API 호출 (날씨, 지도 등)
	코드 실행, 시스템 명령
	로봇/장치 제어

2. How Tool Use Works (일반 흐름)

보통 LLM에게 사용 가능한 tool을 알려주고, LLM이 tool이 필요할 때는 output으로 tool 사용이 필요하다고 알려준다.
이후 코드에서 if 문과 같은 로직에 결려 실제 tool을 호출하고, 결과를 LLM에게 다시 전달한다.

❶ Tools 등록 (System)

사용 가능한 tool 목록과 사용 방법(스키마)을 LLM에게 제공한다.

<Tools List / Schema>

```
[{
  "name": "search_web",
  "description": "웹 검색",
  "parameters": {....}
} , ...]
```

❷ LLM 응답 (의사결정)

사용자 요청을 이해하고, 필요 시 tool 사용을 요청한다.

사용자: 오늘 서울 날씨 어때?

```
TOOL_CALL
{
  "name": "get_weather",
  "arguments": {"location": "Seoul"}
}
```

❸ 코드에서 Tool 호출 (Dispatch)

코드에서 if/else 로직 등으로 tool_call을 감지하고 실제 tool을 호출한다.

```
if response.type == "tool_call":
  tool_name = response.name
  args = response.arguments
  result = call_tool(tool_name, args)
```

❹ Tool 실행 (실제 작업 수행)

외부 시스템에서 작업을 수행하고 결과를 반환한다.

웹 검색 · 날씨 API · 파일 읽기 · 코드 실행 · ...

❺ 결과 반환 및 LLM 응답 (최종 답변 생성)

결과를 LLM에게 작업하고, LLM이 최종 답변을 생성한다.

```
TOOL_RESULT
{
  "temperature": "28°C",
  "condition": "맑음",
  "humidity": "60%"
}
```

서울의 오늘 날씨는 맑고 28°C입니다.

(필요 시 여러 번 반복)

3. MCP (Model Context Protocol)와 Tool Use

MCP는 LLM 애플리케이션이 외부 도구/데이터와 안전하고 표준화된 방식으로 연결되도록 돕는 개방형 프로토콜이다.

MCP의 핵심 역할

- 표준화된 방식으로 Tool 목록, 스키마, 실행을 제공
- 보안 및 권한 관리
- 다양한 도구/데이터 소스를 일관되게 연결
- 확장 가능한 생태계 구성

4. 요약

- LLM이 할 수 없는 일을 Tool을 통해 수행한다.
- LLM에게 사용 가능한 Tool을 알려주고,
  필요할 때 LLM이 tool 사용을 요청한다.
- 코드에서 해당 요청을 감지(if 조건 등)하여
  실제 tool을 호출하고 결과를 다시 LLM에 전달한다.
- MCP는 이러한 Tool 연동을 표준화하고
  안전하게 만들어주는 프로토콜이다.

핵심 메시지 Tool Use는 LLM의 한계를 넘게 해주며, MCP는 그 연결을 표준화하여 더 강력하고 안전한 AI 시스템을 가능하게 한다.

Reflection: Learn from Each Episode, Get Better Over Time

Reflection은 한 Episode(또는 Trial)가 끝날 때마다 결과를 평가하고 원인을 분석하여,
다음 Episode에서 더 나은 전략과 행동을 만들 수 있도록 돕는 과정이다.

1. What is Reflection?

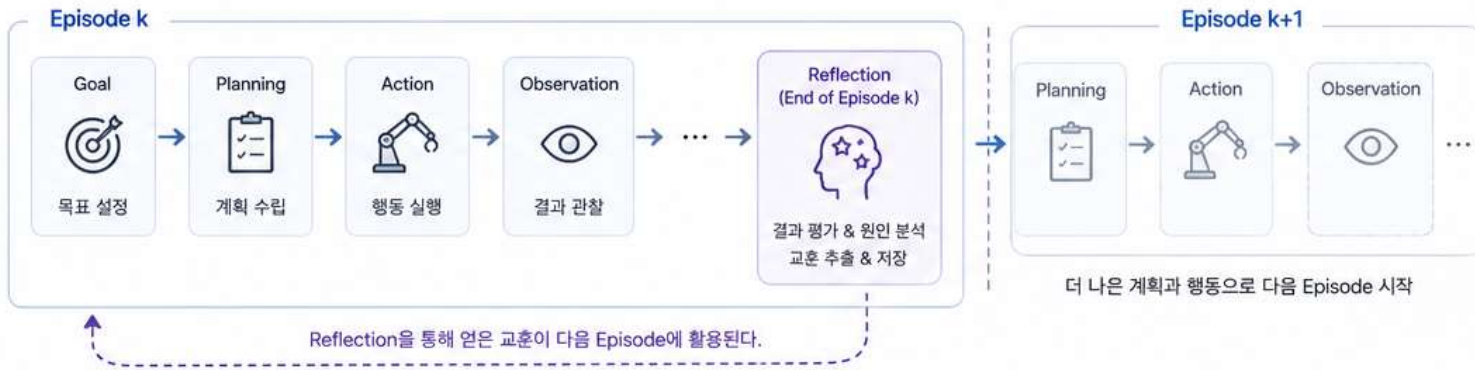
Episode 단위로 수행 결과를 돌아보고,
무엇이 잘 되었고, 무엇이 문제였는지
분석하여 교훈(Insight)을 추출한다.

목적

- 실패 원인 파악 및 개선 방향 도출
- 성공 요인 일반화 및 재사용
- 장기 메모리(Experience / Knowledge) 업데이트
- 다음 Episode의 계획과 행동 품질 향상

2. Reflection Happens at the End of Each Episode

에이전트는 반복되는 Episode(또는 Trial) 단위로 문제를 해결하며, 각 Episode가 끝날 때 Reflection이 일어난다.



3. Reflection Process (Episode 단위)

1 결과 평가

- 목표 달성 여부
- 성능 지표(정확도, 시간 등)
- 제약 조건 만족 여부

2 원인 분석

- 무엇이 잘 되었는가?
- 무엇이 잘못 되었는가?
- 왜 실패/성공했는가?

3 교훈(Insight) 도출

- 핵심 원인 정리
- 개선 아이디어 도출
- 일반화 가능한 교훈 추출

4 지식 저장

- Experience Memory에 저장
- (필요 시) Knowledge / Semantic Memory 업데이트

5 다음 Episode 반영

- 계획 수정
- 전략/파라미터 조정
- 도구 선택 방식 개선

4. Example

Episode k

목표: 건물에서 석민이를 찾아라

결과: 실패 (찾지 못함)

이유: 3층과 4층은 확인했지만, 5층 회의실을 확인하지 못함

지표: 성공률 0%, 탐색 범위 부족

Reflection

- 모든 층/회의실을 체계적으로 탐색하는 전략이 필요
- 탐색 순서 계획을 명시적으로 포함해야 함

Episode k+1

개선: 층별 체크리스트 기반 탐색 전략 추가

결과: 성공 ✅

지표: 성공률 100%

5. Key Takeaways

Episode 단위

Reflection은 연속적인 과정이 아니라, 각 Episode(또는 Trial)가 끝날 때마다 수행된다.

과거에서 배우고 미래에 적용

과거의 경험을 분석하여 얻은 교훈을 다음 Episode의 계획과 행동에 반영한다.

지속적 개선의 핵심

Reflection이 누적될수록 에이전트는 더 똑똑해지고, 더 높은 성공률을 달성한다.

핵심 메시지

Reflection은 '실패를 되돌아보는 것'이 아니라,
'다음 성공을 설계하는 과정'이다.

Introducing Concept of Agentic AI
- Fun Project : Finding Dating Places -

데이트 장소 추천 Agent 시스템 구조

사용자 요청을 이해하고, 과거 경험과 실시간 정보를 활용하여
가장 적합한 데이트 장소 하나를 추천하고,
피드백을 통해 스스로 학습하는 Agent 시스템

Front End (Web)

광안리 근처 야경이 예쁘고 조용한 와인바 | Search

추천 장소

로맨틱 오션 와인바

- 부산 수영구 광안해변로 123
- N 4.5 ★ (리뷰 312)
- G 4.6 ★ (리뷰 201)
- 인스타그램 링크

긍정 리뷰 요약
분위기 좋고 야경이 정말 아름다워요.

부정 리뷰 요약
주말에는 사람이 많을 수 있어요.

추천 이유
야경, 조용한 분위기, 와인 메뉴가 다양,
사용자 선호(대화하기 좋은 장소)와 일치

신뢰도 0.82

✓ Accept (코스에 추가) | ✗ Reject (이유 입력)

데이트 코스 후보 리스트

1. 로맨틱 오션 와인바

Back End (AI Agent)

1. 요청 이해 (NLU & Intent Parsing)

자연어 요청 → 구조화된 SearchIntent 변환

SearchIntent
- location
- place_types
- desired_features
- undesired_features
- time_context
- ...

2. 기억 조회 (Memory Retrieval)

과거 경험과 취향을 조회하여 현재 요청에 활용

- Episodic Memory (과거 SearchEpisode)
- Semantic Memory (사용자 취향/지식)
- Working Memory (현재 작업 정보)

3. 검색 계획 수립 (Search Planning)

검색 전략 및 키워드 생성, 검색 범위 결정

예시 검색 키워드
- 광안리 와인바 야경
- 광안리 조용한 와인바
- 광안리 분위기 좋은 레스토랑
- ...

검색 전략
- 장소 유형 우선순위
- 지역 범위
- 정렬 기준
- 필터 조건

4. 후보 장소 수집 (Data Collection)

여러 플랫폼에서 후보 장소 수집

연결된 데이터 소스
- 네이버 지도 API
- 구글 Places API
- 인스타그램 API
- 날씨 API
- 기타 데이터 소스

5. 정보 통합 (Data Integration)

각 플랫폼의 정보를 통합하고 정규화

UnifiedPlace (통합 스키마)
- place_id
- name
- address
- lat, lng
- category
- naver_rating, naver_cnt
- google_rating, google_cnt
- reviews (pos/neg)
- instagram_links
- ...

6. 후보 평가 및 순위화 (Evaluation & Ranking)

여러 기준으로 평가 후 순위화

평가 기준
- 요청 일치도
- 사용자 취향 일치도
- 리뷰 신뢰도
- SNS 화제성
- 데이트 적합성
- 거절 위험
- 중복 추천 패널티

7. 최종 장소 선택 및 설명 생성 (Selection & Explanation)

최종 1곳을 선택하고 추천 이유와 신뢰도 생성

출력
- 추천 장소 정보
- 추천 이유
- 리뷰 요약
- 관련 인스타그램 링크
- 추천 신뢰도

8. 응답 생성 (Generate Response)

사용자에게 보여줄 응답 생성

응답 패키지
- 장소 정보
- 지도 좌표
- 팡점/리뷰 요약
- 추천 이유
- 신뢰도

9. 피드백 처리 (Feedback Handling)

사용자 Accept/Reject 및 이유 수집

피드백 정보
- decision: Accept / Reject
- reason (선택/필수)
- accept_reason (선택)
- meta (시간, 위치, 날씨 등)

10. Reflection & Memory Update (자기성찰 및 기억 업데이트)

Episode 단위 Reflection 수행 후 기억 업데이트

Reflection 항목
- 성공/실패 원인 분석
- 요청 이해 정확도
- 검색 전략 적절성
- 평가 기준 문제점
- 다음 전략 개선 방향

- Episodic Memory 저장
- Semantic Memory 업데이트

외부 데이터 소스

- 네이버 지도 API
- 구글 Places API
- 인스타그램 API
- 날씨 API
- 교통/길찾기 API
- 기타 데이터 소스

데이터 저장소

- SearchEpisode DB (이력 저장)
- User Profile DB (사용자 설정/프로필)
- Place DB (장소 마스터 데이터)
- Cache / Index (검색 캐시, 임베딩 등)

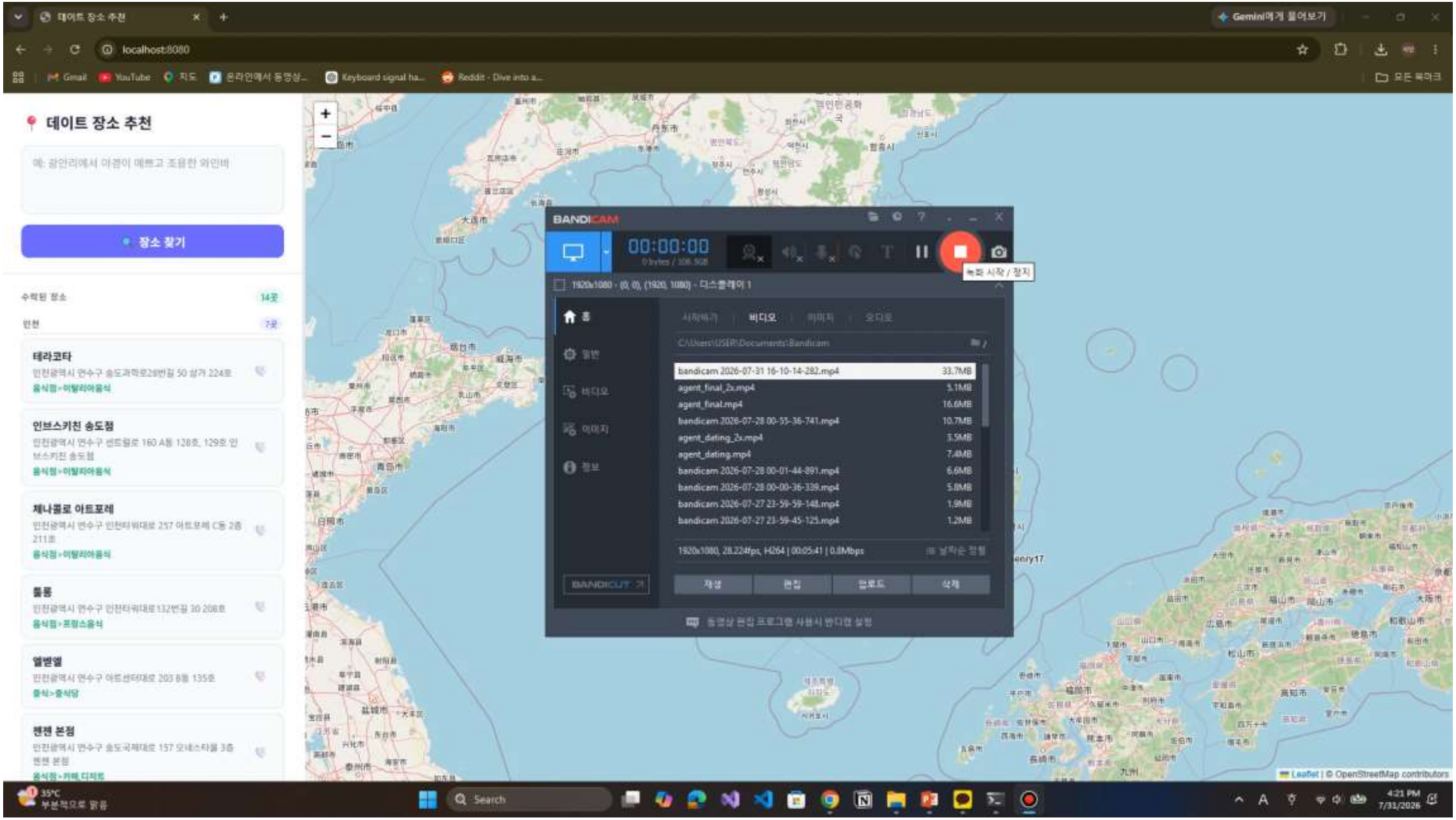
전체 동작 흐름 (요약)

핵심 특징

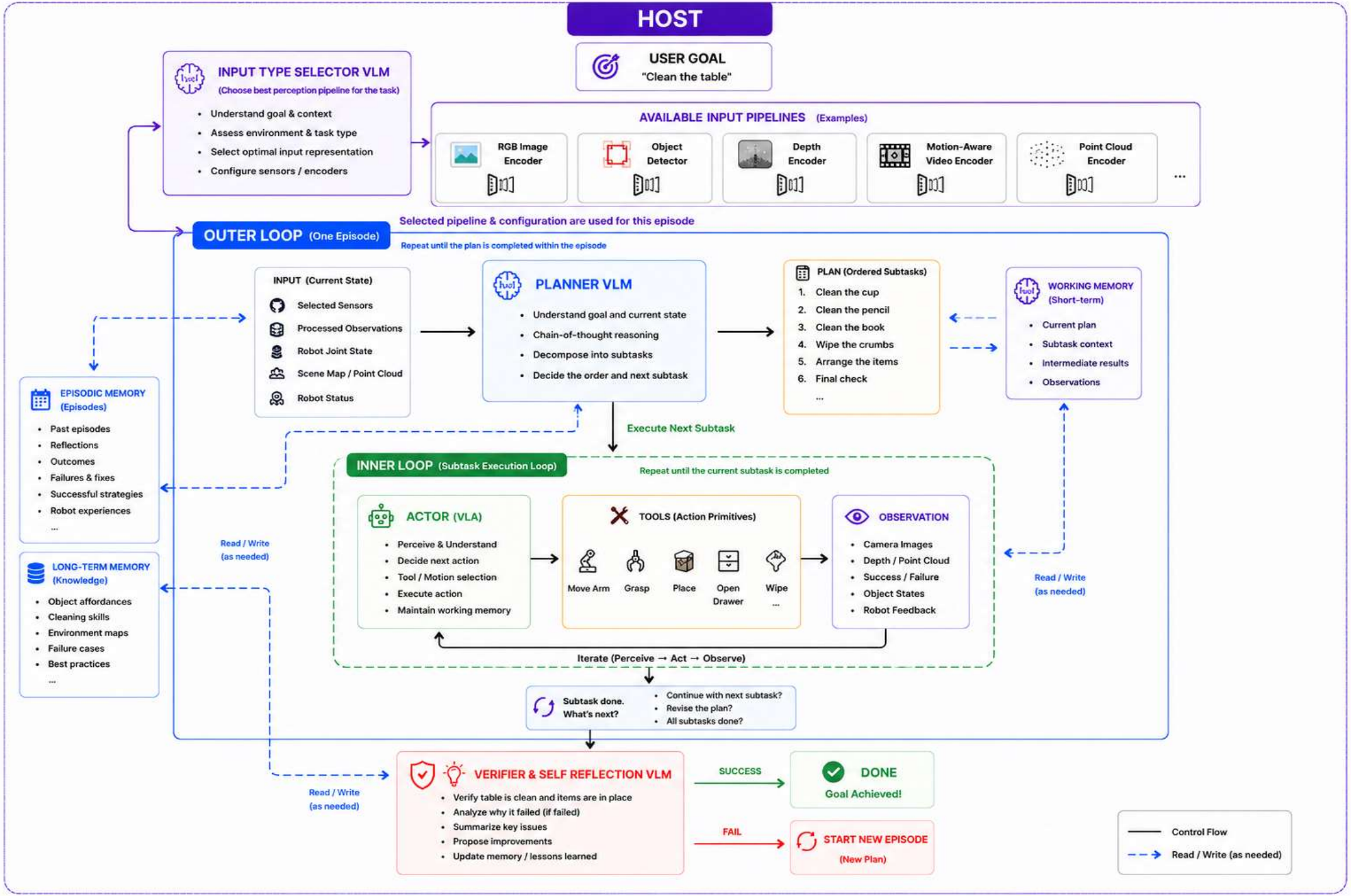
- ✓ 과거 경험(Episode)과 일반화된 취향(Semantic)을 함께 활용
- ✓ 하나의 장소만 추천하여 사용자의 결정 부담을 줄임
- ✓ Reject 이유를 반영하여 다음 추천의 정확도 향상
- ✓ Episode 단위 Reflection으로 지속적으로 전략 개선

범례

- 입력/출력
- 처리 단계
- 메모리 관련
- 저장소
- 외부 시스템

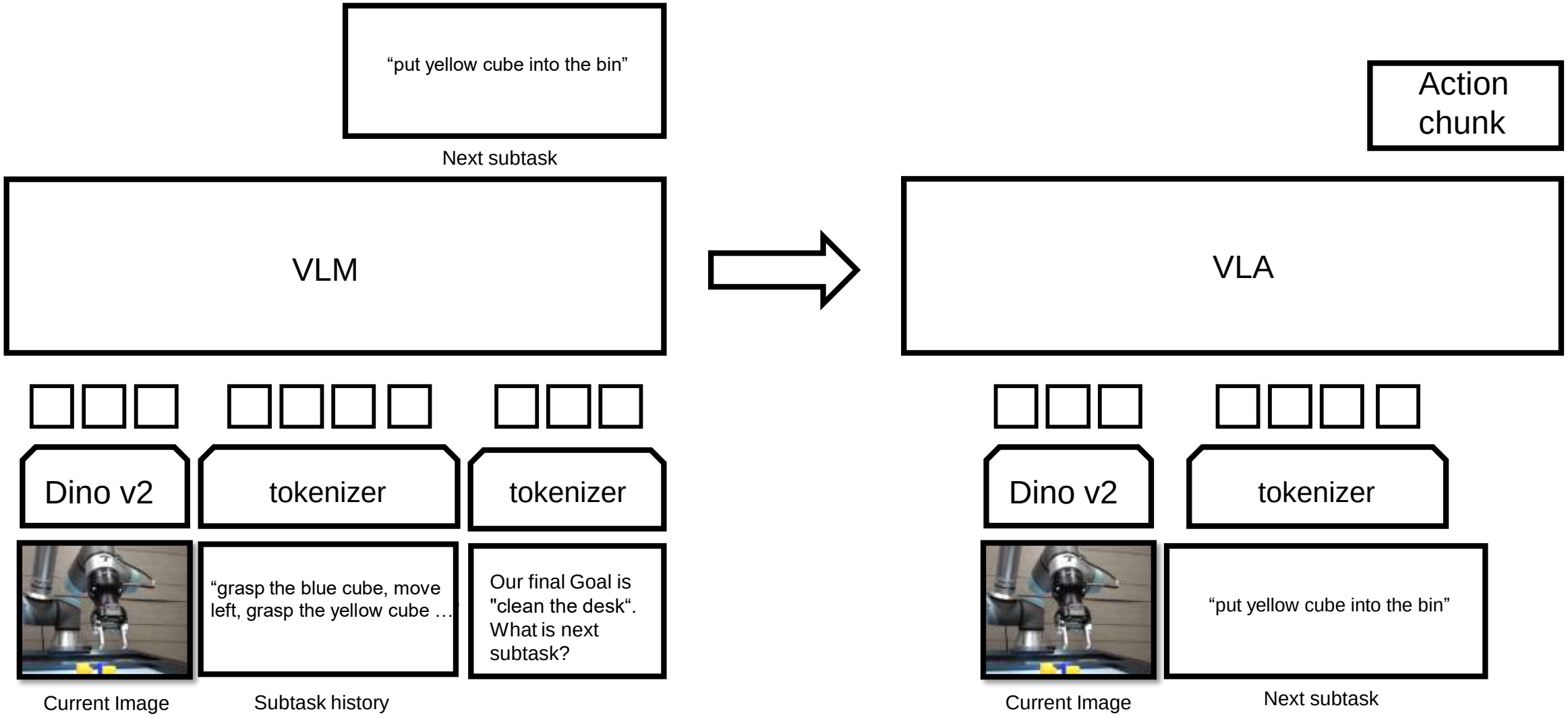

데이트 장소 추천
예: 광안리에서 야경이 예쁘고 조용한 와인바
장소 찾기
수락된 장소 14곳
인천 7곳
테라코타
인천광역시 연수구 송도과학로28번길 50 상가 224호
음식점>이탈리아음식
인브스키친 송도점
인천광역시 연수구 센트럴로 160 A동 128호, 129호 인브스키친 송도점
음식점>이탈리아음식
체나폴로 아트포레
인천광역시 연수구 인천타워대로 257 아트포레 C동 2층 211호
음식점>이탈리아음식
돌풍
인천광역시 연수구 인천타워대로132번길 30 208호
음식점>포장스음식
엘빈엘
인천광역시 연수구 아트센터대로 203 8동 135호
중식>중식당
챈챈 본점
인천광역시 연수구 송도국제대로 157 오네스타워 3층 챈챈 본점
음식점>카페 디저트
BANDICAM
00:00:00
0 bytes / 106.9GB
녹화 시작 / 정지
1920x1080 - (0, 0), (1920, 1080) - 디스플레이 1
홈
일반
비디오
이미지
정보
BANDICUT 기
비디오
C:\Users\USER\Documents\Bandicam
bandicam 2026-07-31 16-10-14-282.mp4 33.7MB
agent_final_2x.mp4 5.1MB
agent_final.mp4 16.6MB
bandicam 2026-07-28 00-55-36-741.mp4 10.7MB
agent_dating_2x.mp4 3.5MB
agent_dating.mp4 7.4MB
bandicam 2026-07-28 00-01-44-891.mp4 6.6MB
bandicam 2026-07-28 00-00-36-339.mp4 5.8MB
bandicam 2026-07-27 23-59-59-148.mp4 1.9MB
bandicam 2026-07-27 23-59-45-125.mp4 1.2MB
1920x1080, 28.224fps, H264 | 00:05:41 | 0.8Mbps
재생
편집
업로드
삭제
Leaflet | © OpenStreetMap contributors
Search
4:21 PM
7/31/2026

Possible Design Choices for Robotic Agents


HOST
USER GOAL
"Clean the table"
INPUT TYPE SELECTOR VLM
(Choose best perception pipeline for the task)
Understand goal & context
Assess environment & task type
Select optimal input representation
Configure sensors / encoders
AVAILABLE INPUT PIPELINES (Examples)
RGB Image Encoder
Object Detector
Depth Encoder
Motion-Aware Video Encoder
Point Cloud Encoder
...
Selected pipeline & configuration are used for this episode
OUTER LOOP (One Episode)
Repeat until the plan is completed within the episode
INPUT (Current State)
Selected Sensors
Processed Observations
Robot Joint State
Scene Map / Point Cloud
Robot Status
PLANNER VLM
Understand goal and current state
Chain-of-thought reasoning
Decompose into subtasks
Decide the order and next subtask
PLAN (Ordered Subtasks)
1. Clean the cup
2. Clean the pencil
3. Clean the book
4. Wipe the crumbs
5. Arrange the items
6. Final check
...
WORKING MEMORY
(Short-term)
Current plan
Subtask context
Intermediate results
Observations
EPISODIC MEMORY
(Episodes)
Past episodes
Reflections
Outcomes
Failures & fixes
Successful strategies
Robot experiences
...
Execute Next Subtask
INNER LOOP (Subtask Execution Loop)
Repeat until the current subtask is completed
ACTOR (VLA)
Perceive & Understand
Decide next action
Tool / Motion selection
Execute action
Maintain working memory
TOOLS (Action Primitives)
Move Arm
Grasp
Place
Open Drawer
Wipe
...
OBSERVATION
Camera Images
Depth / Point Cloud
Success / Failure
Object States
Robot Feedback
Read / Write (as needed)
Read / Write (as needed)
LONG-TERM MEMORY
(Knowledge)
Object affordances
Cleaning skills
Environment maps
Failure cases
Best practices
...
Iterate (Perceive → Act → Observe)
Subtask done. What's next?
Continue with next subtask?
Revise the plan?
All subtasks done?
Read / Write (as needed)
VERIFIER & SELF REFLECTION VLM
Verify table is clean and items are in place
Analyze why it failed (if failed)
Summarize key issues
Propose improvements
Update memory / lessons learned
SUCCESS
DONE
Goal Achieved!
FAIL
START NEW EPISODE
(New Plan)
Control Flow
Read / Write (as needed)

Research Direction

Symbolic Memory : It was an non-optimal agent!



Research Direction

- Symbolic memory를 사용하는 previous approaches도 나름 agent긴 했다.
  - 메모리가 오직 "이전 subtask들의 history"인
- 우리는 이렇게 파면 좋을 것 같다.
  - Demo를 막 돌려보면서 메모리가 " 이전 subtask들의 history " 일 때 생기는 문제 지적
    - 정보가 정제되지 않아서 다음 subtask 예측을 잘 못했다던가…
    - 같은 subtask를 여러 번 실수로 append 해놓았다던가…
- 새로운 memory 방법 제시
  - 현재 env에 대한 state를 정리해 놓은거라던가 (지금까지 버튼을 두번 누름 등등..)
  - Planner를 쓰면서 memory로 다른 type들을 같이 쓴다던가
    - 여기에 더해서 task마다 type을 바꾼다던가 등등 …

Research direction

- 개인적으로는 memory로 자연어만 쓴다고 해도
  - 지금처럼 subtask history를 그냥 저렇게 무식하게 저장하는 것 보다는
  - Task마다 자연어 memory를 task에 맞게 만들게 미리 시켜놓고 (task 시작하기 전에 LLM이 만듦)
    - 컵 2번 쌓는거면 컵 몇번 쌓았는지 저장하는 형식의 메모리 만듦
  - 그 메모리로 task 풀면 성능 오를 것 같음.

Research direction

- 아까 robotics agent그림에서 우리는 outer loop안쪽을 짠다고 생각하면 된다!!
  - Reflecction은 이 연구 잘 되면 나중에 하자
- Inner loop도 잘 짜면 좋을 듯.
  - 지금 사실상 inner loop가 분리가 안되어있는데, 이걸 분리시키는게 훨 안정적일 것 같음.
  - 잡는 중인지 잡았는지 이런것들 을 status로 나타내주는 checker같은걸 뒤에 두면 좋을 것 같음
    - In progress
    - Done

결론은

- 시스템을 향상시킬 수 있는 아이디어는 많음
- 언넝 limitation을 분석하고 그걸 아이디어랑 연결지어서 실험하면 될듯

THE END